

Apunte de Cátedra

ARBOLES

1.- INTRODUCCION

Hasta el momento sólo se han estudiado estructuras de datos lineales estáticas y dinámicas: a un elemento sólo le sigue otro elemento. Al analizar la estructura árbol se introduce el concepto de estructura no-lineal y dinámica de datos más importante en computación. Dinámica, puesto que puede cambiar durante la ejecución de un programa. No lineal, puesto que a cada elemento del árbol pueden seguirle varios elementos.

2.- TERMINOLOGIA FUNDAMENTAL

Un árbol es una estructura jerárquica de una colección de objetos. Es decir, un árbol es una colección de elementos llamados nodos, uno de los cuales se distingue como raíz, junto con una relación (de "paternidad") que impone una estructura jerárquica sobre los nodos. Los árboles genealógicos y los organigramas son ejemplos comunes de árboles.

Formalmente, un árbol se puede definir de manera recursiva (se utiliza la recursión para definir un árbol porque es una característica inherente a los mismos) como:

1. Un solo nodo es, por sí mismo, un árbol. Ese nodo es también la raíz de dicho árbol.
2. Supóngase que n es un nodo y que $A^1, A^2, \dots, A^K$ son árboles con raíces $n^1, n^2, \dots, n^K$, respectivamente. Se puede construir un nuevo árbol haciendo que n se constituya en el padre de los nodos $n^1, n^2, \dots, n^K$. En dicho árbol, n es la raíz y $A^1, A^2, \dots, A^K$ son los subárboles (o árboles hijos) de la raíz. Los nodos $n^1, n^2, \dots, n^K$ reciben el nombre de hijos del nodo n y el nodo n recibe el nombre de padre de dichos nodos.

Una forma particular de árbol es el árbol nulo o vacío, que es un "árbol" sin nodos.

Gráficamente la estructura árbol se puede representar de diferentes formas:

- a) Diagramas de Venn.
- b) Anidación de paréntesis.
- c) Notación decimal de Dewey.
- d) Notación indentada.
- e) Grafos jerárquicos, que es la que más se utiliza.

Como ejemplo se puede considerar el índice de un libro.

T1 (Tema 1)

1.1.- (Pregunta 1 del Tema 1)

1.2.- (Pregunta 2 del Tema 1)

T2 (Tema 2)

2.1.- (Pregunta 1 del Tema 2)

2.1.1.- (Pregunta 1 de la pregunta 1 del Tema 2)

2.1.2.- (Pregunta 2 de la pregunta 1 del Tema 2)

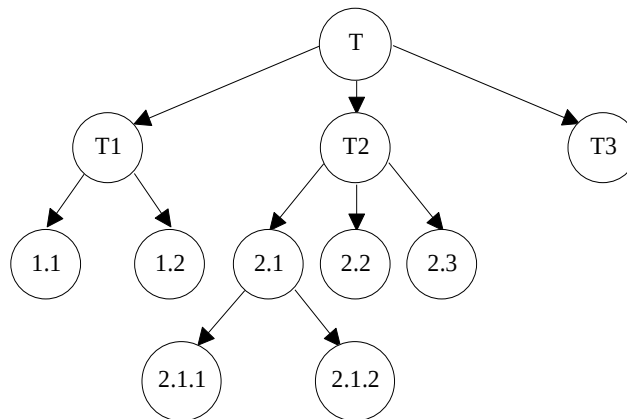
2.2.- (Pregunta 2 del Tema 2)

2.3.- (Pregunta 3 del Tema 2)

T3 (Tema 3)

Tal índice es el árbol que se muestra en la siguiente figura. La relación padre-hijo entre dos nodos se representa por una línea descendente que los une. Normalmente, los árboles se dibujan de arriba hacia abajo, con el padre encima de los hijos. En el ejemplo, la relación de paternidad representa inclusión: el libro está compuesto por los temas 1, 2 y 3.

Apunte de Cátedra



Se llaman hermanos a los árboles hijos del mismo padre. Ejemplo: 2.1, 2.2 y 2.3 son hermanos.

Si $n^1, n^2, \dots, n^K$, es una sucesión de nodos de un árbol tal que n^i es el padre de n^{i+1} para $i = 1, 2, \dots, K-1$, entonces a la sucesión se denomina camino del nodo n^1 al nodo n^K . Ejemplo: T2, 2.1, 2.1.1 es un camino.

La longitud de un camino es el número de nodos del camino menos 1. Por lo tanto, existen caminos de longitud cero, que son aquellos que van de cualquier nodo a sí mismo. Ejemplo: el camino T2, 2.1, 2.1.1 tiene longitud 2.

Si existe un camino de un nodo a a otro b , entonces a es un antecesor de b , y b es un descendiente de a . Cada nodo es a la vez un antecesor y un descendiente de sí mismo. En nuestro ejemplo, los antecesores de 2.1. son el mismo, T2 y Libro. Un antecesor o un descendiente de un nodo que no sea el mismo recibe el nombre de antecesor propio o descendiente propio, respectivamente. En un árbol, la raíz es el único nodo que no tiene antecesores propios. Un nodo sin descendientes propios se denomina hoja o nodo terminal.

Un subárbol de un árbol es un nodo junto con todos sus descendientes. Ejemplo:

Grado de un nodo es el número de subárboles que tiene. En nuestro ejemplo el grado de T2 es 3. Por lo tanto son nodos terminales u hojas los nodos de grado 0.

La altura de un nodo es la longitud del camino más largo de ese nodo a una hoja. En nuestro ejemplo, el nodo T1 tiene altura 1, T2 altura 2 y el nodo T3 altura 0. La altura del árbol es la altura de la raíz.

Nivel o profundidad de un nodo es la longitud del único camino desde la raíz a ese nodo. Por definición la raíz tiene nivel 0. En nuestro ejemplo, el nodo T1 tiene nivel 1, 2.1 nivel 2. La profundidad de un árbol se define como el máximo de los niveles de los nodos del árbol. En el ejemplo la profundidad del árbol es 3.

2.1.- Orden de los nodos

Se habla de un árbol no ordenado cuando explícitamente se ignora el orden de los hijos.

A menudo los hijos de un nodo se ordenan de izquierda a derecha. Así, los dos árboles siguientes son diferentes porque los dos hijos (**b** y **c**) del nodo **a** aparecen en distintos órdenes en los dos árboles.

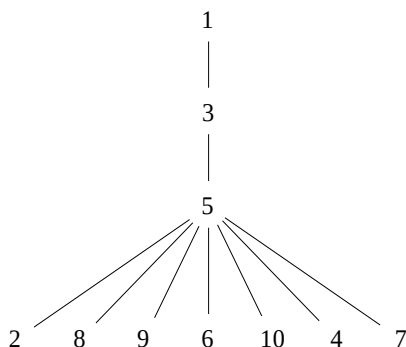


Apunte de Cátedra

El orden de izquierda a derecha de los hermanos se puede extender para comparar dos nodos cualesquiera entre los cuales no exista la relación antecesor-descendiente. La regla es que si a y b son hermanos y a está a la izquierda de b , entonces todos los descendientes de a están a la izquierda de todos los descendientes de b .

Ejemplo 1:

El nodo 8 está a la derecha del nodo 2 y a la izquierda de los nodos 9, 6, 10, 4, 7 y no está a la izquierda ni a la derecha de sus antecesores 1, 3 y 5.



Dado un nodo n , una regla sencilla para determinar qué nodos están a su izquierda y cuáles a su derecha, consiste en dibujar el camino de la raíz a n . Todos los nodos que salen a la izquierda de este camino, y todos sus descendientes, están a la izquierda de n . Los nodos, y sus descendientes, que salen a la derecha, están a la derecha de n .

2.2.- Recorridos de un árbol

Hay varias maneras de recorrer los nodos de un árbol para ser procesados con una cierta operación. Vamos a estudiar dos estrategias para recorrer un árbol y procesar sus nodos:

- Recorrido en profundidad.
- Recorrido en anchura.

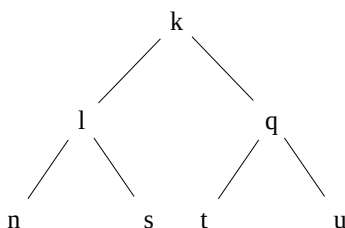
2.2.1.- Recorrido en profundidad

Existen tres formas de recorrer un árbol en profundidad:

- A) Preorden u orden previo.
- B) Inorden u orden simétrico.
- C) Postorden u orden posterior.

Estas tres formas de ordenar un árbol se definen de forma recursiva como sigue:

1. Si el árbol A es nulo, entonces la lista vacía, es el listado de los nodos del árbol A en los órdenes preorden, inorden y postorden.
2. Si el árbol A tiene un solo nodo, entonces ese nodo constituye el listado del árbol A en los tres órdenes (preorden, inorden y postorden).
3. Si el árbol A tiene más de un nodo, es decir, tiene como raíz el nodo n y los subárboles $A^1, A^2, \dots, A^k$ entonces:



Apunte de Cátedra

A) El listado en preorden de los nodos del árbol A está formado por la raíz del árbol A, seguida de los nodos del árbol A^1 en preorden, luego por los nodos de A^2 en preorden y así sucesivamente hasta los nodos de A^K en preorden.

El listado en preorden es: k - l - n - s - q - t - u

B) El listado en inorden de los nodos del árbol A está constituido por los nodos del árbol A^1 en inorden, seguidos de la raíz n y luego por los nodos de A^2 , ..., A^K en inorden.

El listado en inorden es: n - l - s - k - t - q - u

Siempre se debe llegar al último nivel y luego se sube hacia arriba.

C) El listado en postorden de los nodos del árbol A está formado por los nodos del árbol A^1 en postorden, luego los nodos de A^2 en postorden y así sucesivamente hasta los nodos de A^K en postorden y por último la raíz n.

El listado en postorden es: n - s - l - t - u - q - k

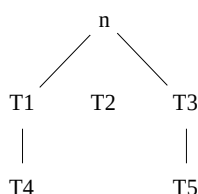
En nuestro ejemplo 1 se lista 1, y luego se llama recursivamente a Preorden del primer subárbol de 1, o sea el subárbol con raíz 2. Este subárbol tiene un solo nodo de manera que simplemente se lista. Luego se sigue con el segundo subárbol de 1, el que tiene raíz 3. Se lista 3 y luego se vuelve a llamar a Preorden con el primer subárbol de 3. Esta llamada origina que se listen 5, 8 y 9, en ese orden. Continuando de esta forma, se obtiene el recorrido en preorden: 1, 2, 3, 5, 8, 9, 6, 10, 4, 7.

En nuestro ejemplo 1 obtendremos el siguiente recorrido en postorden: 2, 8, 9, 5, 10, 6, 3, 7, 4, 1; y en inorden es: 2, 1, 8, 5, 9, 3, 10, 6, 7, 4.

2.2.2.- Recorrido en anchura

Otro posible recorrido de un árbol es un recorrido en anchura, es decir explorando el árbol por niveles, y listando los nodos de izquierda a derecha empezando por el nivel 0, luego el nivel 1, etc.

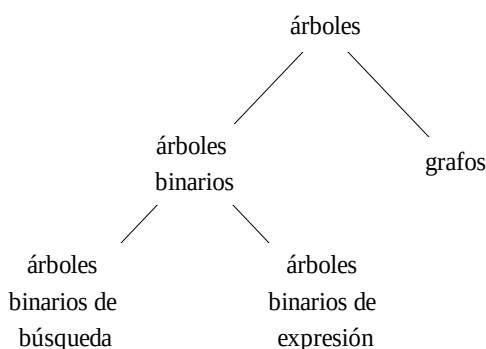
El recorrido en anchura para el árbol: daría la secuencia de nodos: n T^1 T^2 T^3 T^4 T^5



2.3.- Árboles etiquetados

Cuando se asocia una etiqueta, o valor, a cada nodo del árbol, a éste se le denomina árbol etiquetado.

La etiqueta de un nodo no es el nombre del nodo, sino que es información que está incluida en el nodo. Es posible cambiar la etiqueta del nodo sin modificar su nombre.



Apunte de Cátedra

3.- ARBOLES BINARIOS

3.1.- TDA Arbol Binario. Definición

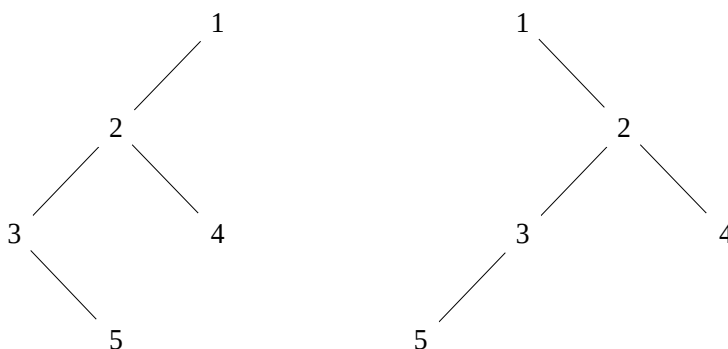
Los árboles binarios son un caso particular de árboles. Un árbol binario es un árbol donde cada nodo tiene como máximo grado 2. Es decir, un árbol binario es ó un árbol vacío, ó un árbol con un solo nodo sin hijos, ó un árbol en que sus nodos tienen un sólo hijo izquierdo, ó un árbol en que sus nodos tienen un sólo hijo derecho, ó un árbol en que sus nodos tienen un hijo izquierdo y un hijo derecho.

En los árboles, vistos anteriormente, los hijos de un nodo se encuentran ordenados de izquierda a derecha. En los árboles binarios esta ordenación es más fuerte, ya que se distingue los hijos de un nodo como izquierdo y derecho y deben estar etiquetados por ello.

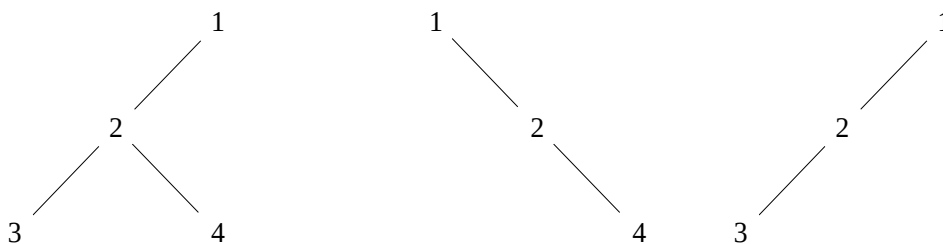
Si se adopta el que los hijos izquierdos se dibujan hacia la izquierda de su padre y los hijos derechos hacia la derecha, los siguientes árboles binarios son distintos.

Ejemplo 2:

En el primer árbol, 2 es el hijo izquierdo de 1 mientras que en el segundo árbol, 2 es el hijo derecho de 1. Los árboles binarios no son directamente comparables con los árboles ordinarios, ya que en éstos no se indica si un hijo es izquierdo o derecho.



En un árbol binario, el borrar un nodo puede dar lugar a dos representaciones distintas, ya que es diferente borrar el hijo derecho o borrar el hijo izquierdo.



Los listados en preorden y postorden de un árbol binario son similares a los de un árbol general, es decir no influye si el hijo es izquierdo o derecho.

Para realizar el listado en inorden sí afecta que el hijo sea izquierdo o derecho. En el caso de que el hijo sea izquierdo, entonces primero se lista dicho hijo y luego el padre. Si el hijo es derecho, entonces primero se lista al padre y luego a este hijo derecho. El listado en inorden de los dos árboles binarios anteriores del ejemplo 2 son: 3, 5, 2, 4, 1 para el primero y 1, 5, 3, 2, 4 para el segundo.

Apunte de Cátedra

3.2.- Implementaciones de Árboles Binarios

Existen dos formas tradicionales para representar un árbol binario en memoria:

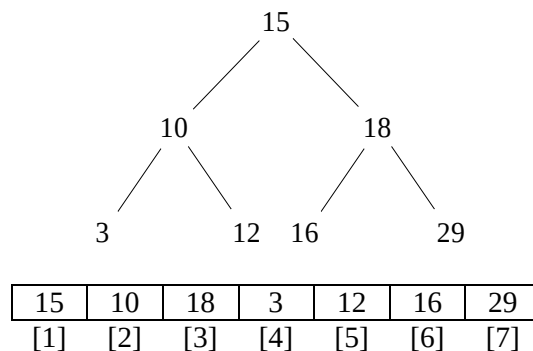
- Mediante arreglos
- Mediante referencias

3.2.1.- Mediante arreglos

Hay una forma de almacenar un árbol binario en un arreglo en la que las relaciones del árbol no están representadas físicamente por campos enlaces sino que están implícitas en los algoritmos que manipulan el árbol almacenado.

Por ejemplo:

El árbol está almacenado en el arreglo por niveles, de izquierda a derecha. El árbol está almacenado con la raíz en Arbol [1] y el último nodo en Arbol [7].



En esta implementación se cumple que para cualquier nodo Arbol[i]:

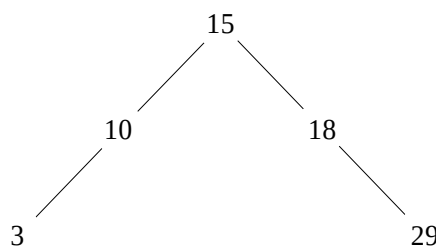
El hijo izquierdo de Arbol [i] está en Arbol [i*2].

El hijo derecho de Arbol [i] está en Arbol [i*2+1].

Para usar esta representación el árbol ha de estar completamente lleno, es decir, todos los nodos han de tener sus dos hijos. Entonces podemos escribir algoritmos para manipular el árbol de esta forma.

También podemos representar las relaciones padre-hijo mediante campos de enlace. Para ello los nombres de los nodos han de poder utilizarse como índices de un arreglo. El arreglo guarda para cada nodo sus hijos izquierdo y derecho. Ahora un árbol no es más que un índice dentro de un arreglo, que indica cual es el nodo que tiene su raíz. El acceso a los subárboles será mediante los índices que marque el nodo. El árbol vacío se representa mediante un índice imposible del arreglo.

Supongamos el siguiente árbol. Su representación podría ser la siguiente:



Apunte de Cátedra

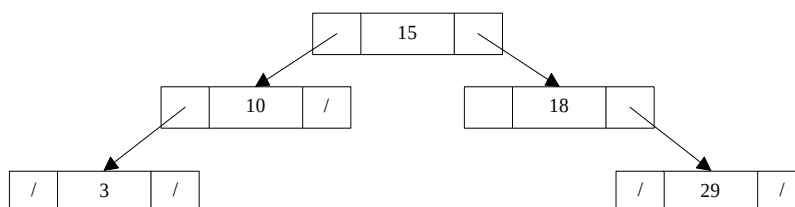
	Hijo Izquierdo	Hijo Derecho	elemento
[1]	0	18	18
[2]	4	1	15
[3]			
[4]	17	0	10
--			
--			
[17]	0	0	3
[18]	0	0	29

Para conocer el padre de cada nodo se le puede añadir a esta estructura un campo más que contenga el nombre del nodo padre del nodo i.

3.2.2.- Mediante referencias

Supone que el árbol se representa por una referencia a un nodo. En él, los hijos son de nuevo árboles, por lo que usaremos la posibilidad de definiciones de estructuras recursivas mediante variables dinámicas. Las declaraciones necesarias serían:

Y el árbol de la figura se representaría:



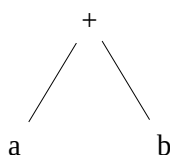
3.3.- Árboles Binarios de Expresión

Un caso particular de los árboles etiquetados lo constituyen los árboles de expresión, utilizados para la representación de expresiones aritméticas. Las reglas para representar una expresión mediante un árbol etiquetado son:

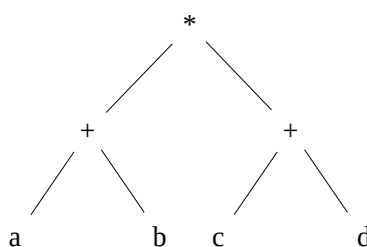
- 1.- Cada hoja está etiquetada con un operando y sólo consta de ese operando.
- 2.- Cada nodo interior está etiquetado con un operador.

Ejemplo 1: la expresión $a+b$ se representaría:

n, n1, n2 son los nombres de los nodos cuyas etiquetas se muestran al lado de los nodos correspondientes. La operación a realizar se pone en el nodo raíz y los operandos en los descendientes de éste.

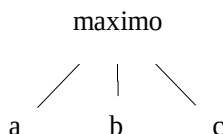


Ejemplo 2: Si la expresión es: $(a + b) * (c + d)$ tendremos:



Apunte de Cátedra

Si en la operación intervienen más de dos operandos, entonces se tendrá más de dos descendientes. Por ejemplo, si la expresión es: máximo (a, b, c), el árbol será:



A menudo, cuando se recorre un árbol en preorden, inorden o postorden, se listan las etiquetas de los nodos en vez de sus nombres.

- Cuando un árbol representa una expresión, el listado en preorden de sus etiquetas da lugar a la forma prefija o polaca de la expresión, en la cual el operador precede a sus operandos izquierdo y derecho. La expresión prefija correspondiente a (E1) q (E2), donde q es un operador binario, es q P1 P2, donde P1 y P2 son las expresiones prefijas correspondientes a E1 y E2.

Se puede observar que en una expresión prefija no se necesitan paréntesis para determinar cuáles son los operandos de cada operación, porque siempre se busca la expresión más corta que se pueda obtener en preorden. Es decir, no hay dudas en la interpretación de lo que se indica.

Por ejemplo, el listado en preorden de las etiquetas de nuestro ejemplo 2 anterior es: * + a b + c d.

- El listado en postorden de las etiquetas de un árbol da lugar a la representación postfija o polaca inversa. La expresión (E1) q (E2) se representa con la expresión postfija como P1 P2 q, donde P1 y P2 son las representaciones postfijas de E1 y E2 respectivamente. Los paréntesis tampoco son necesarios, porque se puede identificar a P1 buscando la expresión más corta de P1 P2 en notación postfija.

En nuestro ejemplo 2, la expresión postfija resultante es: a b + c d + * (P1 será "a b +" porque es la expresión más corta en notación polaca).

- Si se realiza el recorrido de un árbol de expresión en inorden se obtiene la expresión infija, pero sin paréntesis.

Con nuestro ejemplo 2, el listado en inorden resultante sería: a + b * c + d. Se puede observar que es necesario los paréntesis para identificar los operandos de cada operación. Esto se puede realizar añadiendo un paréntesis abierto antes de listar la primera hoja de un subárbol y cerrando el paréntesis después de listar la última hoja del subárbol.

3.4.- Árboles Binarios de Búsqueda

El árbol binario de búsqueda es una estructura sobre la cual se pueden realizar eficientemente las operaciones de búsqueda, inserción y eliminación.

Comparando esta estructura con otra pueden observarse ciertas ventajas:

- en las matrices, si los posibles elementos son muchos, no es práctico emplear los propios elementos del árbol como índices de matrices.
- en las listas, las operaciones de inserción y eliminación se pueden llevar a cabo con facilidad, sin embargo la búsqueda es bastante costosa llevando incluso a recorrer en ocasiones todos los elementos de ella para localizar uno particular.

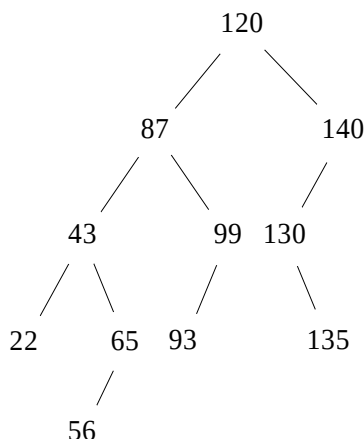
El árbol binario de búsqueda es una estructura de datos básica para almacenar elementos que están clasificados de acuerdo con algún orden lineal.

Se define de la siguiente manera: *Para todo nodo A del árbol debe cumplirse que todos los valores de los nodos del subárbol izquierdo de A deben ser menores al valor del nodo A. De forma similar, todos los valores de los nodos del subárbol derecho de A deben ser mayores al valor del nodo A.*

Apunte de Cátedra

Esta condición, conocida como propiedad del árbol binario de búsqueda, se cumple para todo nodo de un árbol binario de búsqueda, incluyendo la raíz. Esta definición supone que no hay elementos duplicados.

La siguiente figura contiene un árbol binario de búsqueda, correspondiente a la entrada de datos: 120, 87, 43, 99, 140, 22, 65, 56, 93, 130 y 135.



Obsérvese la interesante propiedad de que si se listan los nodos del árbol en Orden Simétrico (InOrden), los elementos quedan clasificados en orden ascendente.

La propiedad del árbol binario de búsqueda hace que sea fácil la localización de un elemento en el árbol. Para determinar si x está en el árbol, primero se compara con el dato r que se encuentre en la raíz. Si $x = r$, el elemento ya está localizado. Si $x < r$, entonces si existe x , sólo puede estar en el subárbol izquierdo de la raíz. De igual modo, si $x > r$, x sólo puede estar en el subárbol derecho de la raíz. El proceso es análogo a una búsqueda binaria en un arreglo.

La inserción es una operación que se puede realizar eficientemente en un árbol binario de búsqueda. La estructura crece conforme se inserten elementos en el árbol. Los pasos que deben realizarse para insertar un elemento son los siguientes:

1. Debe compararse la clave a insertar con la raíz del árbol. Si es mayor, debe avanzarse hacia el subárbol derecho. Si es menor, debe avanzarse hacia el subárbol izquierdo.
2. Repetir sucesivamente el paso 1 hasta que se cumpla alguna de las siguientes condiciones:
 - 2.1 El subárbol derecho es igual a vacío, o el subárbol izquierdo es igual a vacío; en cuyo caso se procederá a insertar el elemento en el lugar que le corresponde.
 - 2.2 La clave que quiere insertarse es igual a la raíz del árbol; en cuyo caso no se realiza la inserción.

La operación de borrado es un poco más complicada que la de inserción. Esta consiste en eliminar un nodo del árbol sin violar los principios que definen justamente un árbol binario de búsqueda. Se deben distinguir los siguientes casos:

1. Si el elemento a borrar es terminal u hoja, simplemente se suprime.
2. Si el elemento a borrar tiene un solo descendiente, entonces tiene que sustituirse por ese descendiente.
3. Si el elemento a borrar tiene los dos descendientes, entonces se tiene que sustituir por el nodo que se encuentra más a la izquierda en el subárbol derecho o por el nodo que se encuentra más a la derecha en el subárbol izquierdo.

Apunte de Cátedra

3.5.- Árboles Binarios Balanceados

Como se puede ver los árboles binarios de búsqueda es una estructura sobre la cual se pueden realizar eficientemente las operaciones de búsqueda, inserción y eliminación. Sin embargo, si el árbol crece o decrece descontroladamente, el rendimiento puede disminuir considerablemente. El caso más desfavorable se produce cuando se inserta un conjunto de claves ordenadas en forma ascendente o descendente.

Con el objeto de mejorar el rendimiento en la búsqueda surgen los árboles balanceados. La idea central de éstos es la de realizar reacomodos o balanceos, después de inserciones o eliminaciones de elementos.

3.5.1.- Árboles Binarios Balanceados AVL

El nombre de AVL es en honor a sus inventores, dos matemáticos rusos, G.M. Adelson_Velskii y E.M. Landis, en el año 1962.

Formalmente se define un árbol balanceado como un árbol binario de búsqueda en el cual se debe cumplir la siguiente condición: Para todo nodo A del árbol, la altura de los subárboles izquierdo y derecho no debe diferir en más de una unidad.

Surge el concepto llamado factor de equilibrio (FE) de un nodo como la altura del subárbol derecho menos la altura del subárbol izquierdo. Los valores que puede tomar FE son -1, 0, 1. Si FE llegara a tomar los valores -2 o 2 entonces debería reestructurarse el árbol.

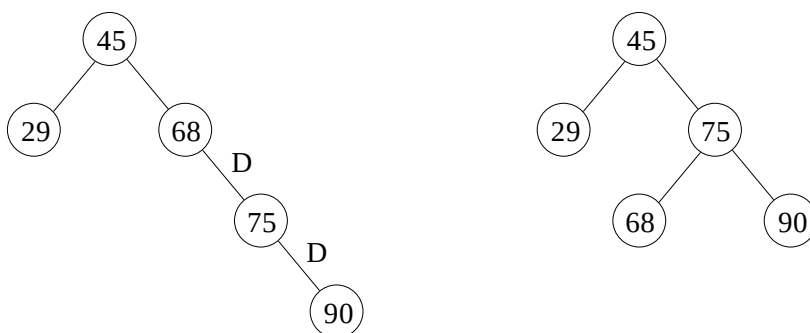
La reestructuración se efectúa cuando al regresar por el camino de búsqueda después de una inserción o una supresión se comprueba que la condición del FE de algún nodo se ha violado. El proceso termina al llegar a la raíz del árbol.

Reestructurar el árbol significa rotar los nodos del mismo. La rotación puede ser simple o compuesta. El primer caso involucra dos nodos y el segundo caso afecta a tres. Si la rotación es simple puede realizarse por las ramas derechas o por las ramas izquierdas.

Si la rotación es compuesta puede realizarse por las ramas derecha e izquierda o por las ramas izquierda y derecha.

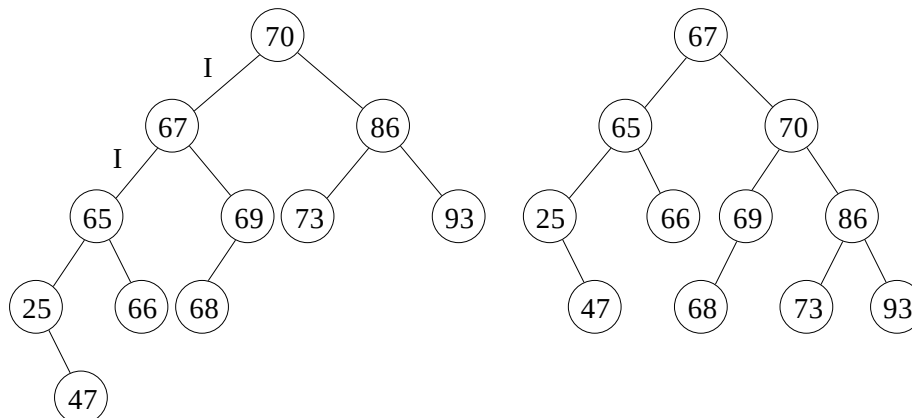
3.5.1.1.- Inserción en Árboles Binarios Balanceados AVL

Si la rotación es DD los nodos rotan en sentido contrario a las agujas del reloj pasando el nodo central como raíz y el movimiento de referencias es el siguiente:

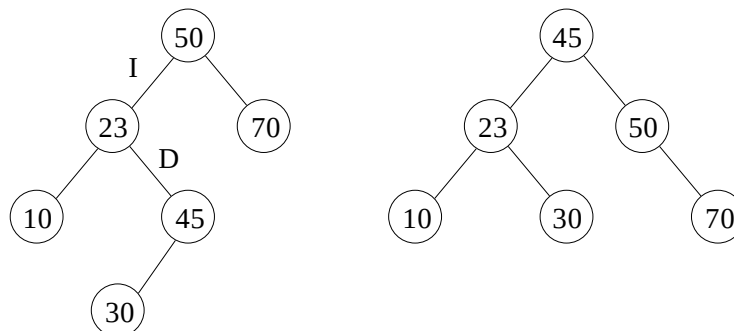


Si la rotación es II los nodos rotan en el sentido de las agujas del reloj pasando el nodo central a nodo raíz y el movimiento de referencias es el siguiente:

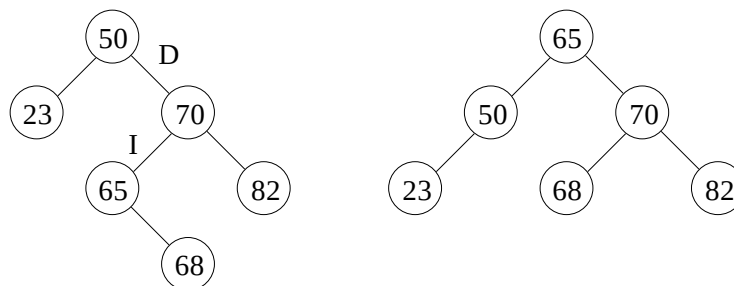
Apunte de Cátedra



Si la rotación es ID el movimiento de referencias es el siguiente:



Si la rotación es DI el movimiento de referencias es el siguiente:



3.5.1.2.- Borrado en Árboles Binarios Balanceados AVL

La operación de borrado es un poco más compleja que la de inserción. Utiliza el mismo algoritmo de borrado que en los árboles binarios de búsqueda y las mismas operaciones de reacomodo que se utilizan en el algoritmo de inserción en árboles balanceados.

Para eliminar un nodo en un árbol balanceado lo primero que debe hacerse es localizar su posición en el árbol. Se elimina siguiendo los criterios establecidos para los árboles binarios de búsqueda y se regresa por el camino de búsqueda calculando el FE de los nodos visitados. Si en alguno de los nodos se viola el criterio de equilibrio, entonces debe reestructurarse el árbol. El proceso termina cuando se llega a la raíz del árbol.

Cabe aclarar que, mientras en el algoritmo de inserción una vez que era efectuada una rotación podía detenerse el proceso, en este algoritmo debe continuarse puesto que se puede producir más de una rotación en el camino hacia atrás.

Apunte de Cátedra

3.5.1.3.- Implementación de Árboles Binarios Balanceados AVL

```
class AVLTree
{
    AVLNode root;           //raíz del árbol
    boolean grown, shrunk, found; //variables artificiales para los métodos

    AVLTree()
    {
        //constructor para un árbol vacío
        root=null;
        grown=false;
        shrunk=false;
        found=false;
    }

    boolean search(int c)
    {
        //busca el contenido c en el árbol
        AVLNode n=root;
        while(n!=null)
        {
            if(c==n.content)
                return true;
            if(c<n.content)
                n=n.left;
            else
                n=n.right;
        }
        return false;
    }

    void balanceError(AVLNode n)
    {
        System.out.println("Error en el valor de equilibrio: "+ n.balance+" y contenido: "+n.content);
    }

    boolean insert(int c)
    {
        //inserta el contenido c en el árbol
        found=false;
        grown=true;           //por defecto el TB más bajo crecerá
        root=insert(root, c);
        return !found;
    }

    AVLNode insert(AVLNode n, int c)
    {
        {
            if(n==null)
                return new AVLNode(c);
            if(c==n.content)
            {
                found=true;
                grown=false;
            }
            else
            {
                if(c<n.content)
                {
                    n.left=insert(n.left, c);
                    if(grown)
                        n.balance--;           //el balance se inclina hacia la izquierda
                }
                else
                {
                    n.right=insert(n.right, c);
                    if(grown)
                        n.balance++;           //el balance se inclina hacia la derecha
                }
                switch(n.balance)
                {
                    case -2: {
                        if(n.left.balance==+1)

```

Apunte de Cátedra

```

        rotateLeft(n.left);
        rotateRight(n);
        grown=false;
        break;
    }
    case -1: break;
    case 0: {
        grown=false;
        break;
    }
    case +1: break;
    case +2: {
        if(n.right.balance==1)
            rotateRight(n.right);
        rotateLeft(n);
        grown=false;
        break;
    }
    default: {
        balanceError(n);
        break;
    }
}

return n;
}

void rotateRight(AVLNode n)
{
    //rotación simple a la derecha
    AVLNode m=n.left;
    int cc=n.content;
    n.content=m.content;
    m.content=cc;
    n.left=m.left;
    m.left=m.right;
    m.right=n.right;
    n.right=m;
    int bm=1 + Math.max(-m.balance, 0) + n.balance;
    int bn=1 + m.balance + Math.max(0, bm);
    n.balance=(byte)bn;
    m.balance=(byte)bm;
}

void rotateLeft(AVLNode n)
{
    //rotación simple a la izquierda
    AVLNode m=n.right;
    int cc=n.content;
    n.content=m.content;
    m.content=cc;
    n.right=m.right;
    m.right=m.left;
    m.left=n.left;
    n.left=m;
    int bm=-1 + Math.max(+m.balance, 0) - n.balance;
    int bn=-1 - m.balance + Math.max(0, -bm);
    n.balance=(byte)bn;
    m.balance=(byte)bm;
}

int height()
{
    //devuelve la altura
    int h=0;
    AVLNode n=root;
    while(n!=null)
    {
        h++;
        if(n.balance>0)
        {
            h+=n.balance;
            n=n.left;
        }
    }
}

```

Apunte de Cátedra

```

        else
        {
            h-=n.balance;
            n=n.right;
        }
    }
    return h;
}

int internal()
{
    //trayectoria interna que alarga
    return internal(root, 0);
}

int internal(AVLNode n, int h)
{
    if(n==null)
        return 0;
    else
    {
        h++;
        return h + internal(n.left, h) + internal(n.right, h);
    }
}

boolean delete(int c)
{
    //borra o elimina el contenido c del árbol
    found=true;
    shrunk=true;
    root=delete(root, c);
    return found;
}

AVLNode delete(AVLNode n, int c)
{
    if(n==null)
    {
        found=false;
        shrunk=false;
        return n;
    }
    if(c==n.content)
    {
        if(n.left==null)
            return n.right;
        if(n.right==null)
            return n.left;
        n.content=c=minValue(n.right);
    }
    if(c<n.content)
    {
        n.left=delete(n.left, c);
        if(shrunk)
            n.balance++;
    }
    else
    {
        n.right=delete(n.right, c);
        if(shrunk)
            n.balance--;
    }
    switch(n.balance)
    {
        case -2: {
            switch(n.left.balance)
            {
                case +1: {
                    rotateLeft(n.left);
                    break;
                }
                case 0: {

```

Apunte de Cátedra

```

        shrunk=false;
        break;
    }
    case -1: break;
    default: {
        balanceError(n.left);
        break;
    }
    }
    rotateRight(n);
    break;
}
case -1: {
    shrunk=false;
    break;
}
case 0: break;
case +1: {
    shrunk=false;
    break;
}
case +2: {
    switch(n.right.balance)
    {
        case -1: {
            rotateRight(n.right);
            break;
        }
        case 0: {
            shrunk=false;
            break;
        }
        case +1: break;
        default: {
            balanceError(n.right);
            break;
        }
    }
    rotateLeft(n);
    break;
}
default: {
    balanceError(n);
    break;
}
}
return n;
}

int minValue(AVLNode n)
{
    //determina el valor más pequeño en el subárbol n
    while(n.left!=null)
        n=n.left;
    return n.content;
}

}

class AVLNode
{
    int content;        //el contenido aquí es entero
    byte balance;       //para los valores -2, -1, 0, 1, +2
    AVLNode left;       //sucesor izquierdo
    AVLNode right;      //sucesor derecho

    AVLNode(int c)
    {
        //constructor para los nuevos nodos
        content=c;
        balance=0;
        left=null;
        right=null;
    }
}

```